Conditional Operators - If Else - Comparison Operators

Conditional Operators

What are Conditional Operators?

Conditional operators allow you to execute different code blocks based on whether certain conditions are true or false. They help control the flow of your program.

Basic Conditional Structure

Simple if Statement

```
let a = 30;
let b = 20;

if (a > b) {
    console.log('pass');
} else {
    console.log('fail');
}

```

How it works:

- If the condition (a > b) is true, execute the first block
- If the condition is false, execute the else block

Dead Code (Unreachable Code)

```
// This code will never execute
if (false) {
    console.log('bye!!');  // Dead code - unreachable
}

```

Important: Code inside an if (false) block is called "dead code" because it will never run. This should be avoided in production code.

Common Mistake: Multiple Independent if Statements

```
// ❌ WRONG APPROACH - Bug + Performance Issue
let browser = 'chrome';

if (browser === 'chrome') {
    console.log('launch chrome');
}
if (browser === 'firefox') {
    console.log('launch firefox');
}
if (browser === 'edge') {
    console.log('launch edge');
}
if (browser === 'safari') {
    console.log('launch safari');

```

```
16  }
17  else {
18      console.log('plz pass the right browser....');
19  }
```

Problems with this approach:

1. **Bug**: The else only applies to the last if statement
2. **Performance**: All conditions are checked even after finding a match
3. **Logic Error**: Multiple conditions might execute

Correct Approach: if-else if Chain

```
// ✅ CORRECT APPROACH
let browserName = 'chrome';

if (browserName === 'chrome') {
    console.log('launch chrome');
}
else if (browserName === 'firefox') {
    console.log('launch firefox');
}
else if (browserName === 'edge') {
    console.log('launch edge');
}
else if (browserName === 'safari') {
    console.log('launch safari');
}
else {
    console.log('plz pass the right browser....');
}

```

Benefits:

- **Exclusive execution**: Only one block executes
- **Better performance**: Stops checking once a condition is met
- **Proper fallback**: The final else catches all unmatched cases

Note: Even with if-else if, there's still a performance consideration. If browserName = 'safari', JavaScript will still check chrome, firefox, and edge conditions before reaching safari. For better performance with many conditions, consider using switch statements or object lookups.

Boolean Conditions

```
let isElementDisplayed = false;

```

```
if (isElementDisplayed) {
    console.log('click on the element');
}
else {
    console.log('wait for the element');
}

```

Note: You can use boolean variables directly in conditions without comparing to true or false.

Comparison Operators

Common operators used in conditions:

Operator	Description	Example
==	Loose equality	10 == "10" → true
===	Strict equality	10 === "10" → false
!=	Loose inequality	10 != "5" → true
!==	Strict inequality	10 !== "10" → true
>	Greater than	30 > 20 → true
<	Less than	20 < 30 → true
>=	Greater than or equal	30 >= 30 → true
<=	Less than or equal	20 <= 30 → true

Performance Considerations

Sequential Checking in if-else if Chains

```
let browserName = 'safari';

if (browserName === 'chrome') {        // ❌ Checked first
    console.log('launch chrome');
}
else if (browserName === 'firefox') {  // ❌ Checked second
    console.log('launch firefox');
}
else if (browserName === 'edge') {     // ❌ Checked third
    console.log('launch edge');
}
else if (browserName === 'safari') {   // ✅ Finally matches (4th check)
    console.log('launch safari');
}
else {
    console.log('plz pass the right browser....');
}

```

Performance Issue: Even with if-else if, if the matching condition is at the end, all previous conditions are still checked sequentially.

Alternative Solutions for Better Performance

Option 1: Switch Statement

```
let browserName = 'safari';
```

```
2
3  switch (browserName) {
4      case 'chrome':
5          console.log('launch chrome');
6          break;
7      case 'firefox':
8          console.log('launch firefox');
9          break;
10     case 'edge':
11         console.log('launch edge');
12         break;
13     case 'safari':
14         console.log('launch safari');
15         break;
16     default:
17         console.log('plz pass the right browser....');
18 }
```

Best Practices

1. Order Conditions by Frequency

```
1  // ✅ Put most common conditions first
2  if (browser === 'chrome') {        // Most common first
3      console.log('launch chrome');
4  }
5  else if (browser === 'firefox') {
6      console.log('launch firefox');
```

```
}
else if (browser === 'safari') {
    console.log('launch safari');
}
else if (browser === 'edge') {     // Least common last
    console.log('launch edge');
}

```

2. Use Switch for Multiple Equal Comparisons

```
// ✅ Good - only one condition can be true
if (score >= 90) {
    console.log('A grade');
}
else if (score >= 80) {
    console.log('B grade');
}
else {
    console.log('C grade');
}

```

3. Use if-else if for Mutually Exclusive Conditions

```
// ✅ Good - only one condition can be true
if (score >= 90) {
    console.log('A grade');
}
else if (score >= 80) {
```

```
    console.log('B grade');
}
else {
    console.log('C grade');
}

```

4. Use Strict Equality (===)

```
// ✅ Recommended
if (browser === 'chrome') {
    // code
}

// ❌ Can cause unexpected behavior
if (browser == 'chrome') {
    // code
}

```

5. Avoid Dead Code

```
// ❌ Avoid this
if (false) {
    console.log('This will never run');
}

// ✅ Remove dead code or use meaningful conditions
if (isUserLoggedIn) {
    console.log('Welcome back!');
```

```
}

```

6. Use Descriptive Variable Names

```
// ✅ Good
let isElementDisplayed = false;
if (isElementDisplayed) {
    // code
}

// ❌ Less clear
let flag = false;
if (flag) {
    // code
}

```

Common Pitfalls

1. **Forgetting else if**: Using multiple if statements instead of else if
2. **Dead code**: Writing conditions that can never be true
3. **Performance issues**: Checking all conditions when only one should match
4. **Logic errors**: Incorrect else placement with multiple if statements

Memory Tips

- **if-else if-else**: Think of it as a "decision tree" - go down the path until you find a match
- **Multiple ifs**: Think of it as "multiple questions" - all questions are asked regardless of answers

- **Dead code**: If the condition is always false, the code is "dead" (won't execute)

Real-World Example

```
let userRole = 'admin';

if (userRole === 'admin') {
    console.log('Full access granted');
}
else if (userRole === 'user') {
    console.log('Limited access granted');
}
else if (userRole === 'guest') {
    console.log('Read-only access granted');
}
else {
    console.log('Access denied');
}

```

This pattern is commonly used in authentication, form validation, and decision-making logic in web applications.